

Arquitecturas para interfaces de usuario

Three horizontal bars of different shades of green and blue, stacked on top of each other, spanning the width of the slide.

Programación III - UNGS

Interfaces de usuario

- Las **interfaces de usuario** (GUI) involucran código que, dependiendo de la tecnología, puede ser extenso.

Interfaces de usuario

- Las **interfaces de usuario** (GUI) involucran código que, dependiendo de la tecnología, puede ser extenso.
- El código de la GUI debe ser tratado con los mismos estándares de calidad que el código de negocio.

Interfaces de usuario

- Las **interfaces de usuario** (GUI) involucran código que, dependiendo de la tecnología, puede ser extenso.
- El código de la GUI debe ser tratado con los mismos estándares de calidad que el código de negocio.
- Desde acciones en la GUI se accede a la funcionalidad de negocio.

Interfaces de usuario

- Las **interfaces de usuario** (GUI) involucran código que, dependiendo de la tecnología, puede ser extenso.
- El código de la GUI debe ser tratado con los mismos estándares de calidad que el código de negocio.
- Desde acciones en la GUI se accede a la funcionalidad de negocio.
 1. Esto tiene el **riesgo** de incluir inteligencia de negocio en el código encargado de la interfaz (por qué es un riesgo?).

Interfaces de usuario

- Las **interfaces de usuario** (GUI) involucran código que, dependiendo de la tecnología, puede ser extenso.
- El código de la GUI debe ser tratado con los mismos estándares de calidad que el código de negocio.
- Desde acciones en la GUI se accede a la funcionalidad de negocio.
 1. Esto tiene el **riesgo** de incluir inteligencia de negocio en el código encargado de la interfaz (por qué es un riesgo?).
 2. Es importante mantener separado el código encargado de la interfaz del código encargado de la funcionalidad de negocio.

Separated presentation

- **Principio:** Se debe seguir estrictamente el principio de **separated presentation** al momento de diseñar el código de las interfaces.

[Separated presentation] is a form of layering, where we keep presentation code and domain code in separate layers with the domain code unaware of presentation code.

Presentation code would manipulate GUI widgets and structures in a rich client application, HTTP headers and HTML in a web application, or command line arguments and print statements in a command line application. We then divide the application into two logical modules with all the presentation code in one module and the rest in another module.

Further layering is often used to separate data source code from domain (business logic), and to separate the domain using a service layer.

Martin Fowler, 2006.

Arquitectura en tres capas

- Una **arquitectura multi-capa** es una arquitectura de software dividida en niveles, desde la interfaz de usuario hasta el almacenamiento de los datos.

Arquitectura en tres capas

- Una **arquitectura multi-capa** es una arquitectura de software dividida en niveles, desde la interfaz de usuario hasta el almacenamiento de los datos.
- El modelo más habitual es contar con **tres capas** o niveles:

Arquitectura en tres capas

- Una **arquitectura multi-capa** es una arquitectura de software dividida en niveles, desde la interfaz de usuario hasta el almacenamiento de los datos.
- El modelo más habitual es contar con **tres capas** o niveles:
 1. **Nivel de presentación:** Contiene la interfaz de usuario y no realiza procesamiento de información “de negocio”. Sólo realiza el procesamiento necesario para presentar adecuadamente la información.

Arquitectura en tres capas

- Una **arquitectura multi-capa** es una arquitectura de software dividida en niveles, desde la interfaz de usuario hasta el almacenamiento de los datos.
- El modelo más habitual es contar con **tres capas** o niveles:
 1. **Nivel de presentación:** Contiene la interfaz de usuario y no realiza procesamiento de información “de negocio”. Sólo realiza el procesamiento necesario para presentar adecuadamente la información.
 2. **Nivel de aplicación o de negocio:** Contiene la funcionalidad del sistema. En un diseño orientado a objetos, incluye las clases de negocio y su implementación.

Arquitectura en tres capas

- Una **arquitectura multi-capas** es una arquitectura de software dividida en niveles, desde la interfaz de usuario hasta el almacenamiento de los datos.
- El modelo más habitual es contar con **tres capas** o niveles:
 1. **Nivel de presentación:** Contiene la interfaz de usuario y no realiza procesamiento de información “de negocio”. Sólo realiza el procesamiento necesario para presentar adecuadamente la información.
 2. **Nivel de aplicación o de negocio:** Contiene la funcionalidad del sistema. En un diseño orientado a objetos, incluye las clases de negocio y su implementación.
 3. **Nivel de datos:** Contiene el almacenamiento de la información y la funcionalidad asociada con el acceso a la información.

Arquitecturas de GUIs

- Es importante adoptar una arquitectura que permita cumplir el principio de *separated presentation*. Algunas opciones:

Arquitecturas de GUIs

- Es importante adoptar una arquitectura que permita cumplir el principio de *separated presentation*. Algunas opciones:
 1. Forms and controls.

Arquitecturas de GUIs

- Es importante adoptar una arquitectura que permita cumplir el principio de *separated presentation*. Algunas opciones:
 1. Forms and controls.
 2. Model-View-Controller.

Arquitecturas de GUIs

- Es importante adoptar una arquitectura que permita cumplir el principio de *separated presentation*. Algunas opciones:
 1. Forms and controls.
 2. Model-View-Controller.
 3. Model-View-Presenter.

Forms and controls

- Las interfaces de usuario contienen **controles visuales** genéricos, que en general tienen la capacidad de almacenar información sencilla.

Forms and controls

- Las interfaces de usuario contienen **controles visuales** genéricos, que en general tienen la capacidad de almacenar información sencilla.
- En la arquitectura **forms and controls**, se tienen interfaces visuales (llamados *forms*, *frames*, etc.) con controles visuales, y las interfaces realizan llamados al código de negocio.

Forms and controls

- Las interfaces de usuario contienen **controles visuales** genéricos, que en general tienen la capacidad de almacenar información sencilla.
- En la arquitectura **forms and controls**, se tienen interfaces visuales (llamados *forms*, *frames*, etc.) con controles visuales, y las interfaces realizan llamados al código de negocio.
- El código de interfaz realiza llamados al código de negocio, para ...

Forms and controls

- Las interfaces de usuario contienen **controles visuales** genéricos, que en general tienen la capacidad de almacenar información sencilla.
- En la arquitectura **forms and controls**, se tienen interfaces visuales (llamados *forms*, *frames*, etc.) con controles visuales, y las interfaces realizan llamados al código de negocio.
- El código de interfaz realiza llamados al código de negocio, para ...
 1. ... ejecutar acciones solicitadas por el usuario, y

Forms and controls

- Las interfaces de usuario contienen **controles visuales** genéricos, que en general tienen la capacidad de almacenar información sencilla.
- En la arquitectura **forms and controls**, se tienen interfaces visuales (llamados *forms*, *frames*, etc.) con controles visuales, y las interfaces realizan llamados al código de negocio.
- El código de interfaz realiza llamados al código de negocio, para ...
 1. ... ejecutar acciones solicitadas por el usuario, y
 2. ... actualizar los controles visuales cuando cambia el estado del sistema.

Forms and controls

- Las interfaces de usuario contienen **controles visuales** genéricos, que en general tienen la capacidad de almacenar información sencilla.
- En la arquitectura **forms and controls**, se tienen interfaces visuales (llamados *forms*, *frames*, etc.) con controles visuales, y las interfaces realizan llamados al código de negocio.
- El código de interfaz realiza llamados al código de negocio, para ...
 1. ... ejecutar acciones solicitadas por el usuario, y
 2. ... actualizar los controles visuales cuando cambia el estado del sistema.
- En ningún momento el código de negocio debe llamar al código de la interfaz! Más aún, el código de negocio no debe conocer la **identidad** ni la **tecnología** utilizada por la interfaz.

Forms and controls

- **Ejemplo:** Form para cargar un nuevo pedido.

The image shows a smartphone screen with a form titled "Editar pedido". The form contains the following fields and values:

Editar pedido	
Entrega:	8/9/17
Cliente:	MC PAPERS ARGENTINA SOCIEDAD ANI
Producto:	M039-00000005
Cantidad:	12000
Descuento:	3.00 %

At the bottom right of the form are two red buttons: "Crear" and "Volver".

Forms and controls

- **Ejemplo:** Form para cargar un nuevo pedido.



The image shows a smartphone screen with a form titled "Editar pedido". The form contains the following fields and values:

Editar pedido	
Entrega:	8/9/17
Cliente:	MC PAPERS ARGENTINA SOCIEDAD ANI
Producto:	M039-00000005
Cantidad:	12000
Descuento:	3.00 %

At the bottom right of the form are two red buttons: "Crear" and "Volver".

- El usuario carga los datos del pedido, que se guarda con "Crear".

Forms and controls

- **Ejemplo:** Form para cargar un nuevo pedido.



The image shows a smartphone screen with a form titled "Editar pedido". The form contains the following fields and values:

Editar pedido	
Entrega:	8/9/17
Cliente:	MC PAPERS ARGENTINA SOCIEDAD ANI
Producto:	M039-00000005
Cantidad:	12000
Descuento:	3.00 %

At the bottom right of the form are two red buttons: "Crear" and "Volver".

- El usuario carga los datos del pedido, que se guarda con "Crear".
- Cuando se modifica la cantidad, se calcula automáticamente el descuento.

Forms and controls

- Capturamos el evento asociado con el botón “Crear” y llamamos al código de negocio.

Forms and controls

- Capturamos el evento asociado con el botón “Crear” y llamamos al código de negocio.

```

1  Button crear = new Button(" Crear" );
2  crear.addActionListener(new ActionListener<ActionEvent>()
3  {
4      @Override
5      public void actionPerformed(ActionEvent evt)
6      {
7          Date entrega = pickerEntrega.getDate();
8          Cliente cliente = (Cliente) comboCliente.getSelectedItem();
9          Producto producto = (Producto) comboProducto.getSelectedItem();
10         double cantidad = Double.parseDouble(textCantidad.getText());
11
12         pedidos.crearPedido(cliente, producto, entrega, cantidad);
13     }
14 });
  
```

Forms and controls

- Cuando se modifica la cantidad, queremos mostrar el porcentaje de descuento. **Nunca** se debe hacer esto! (por qué?)

Forms and controls

- Cuando se modifica la cantidad, queremos mostrar el porcentaje de descuento. **Nunca** se debe hacer esto! (por qué?)

```

1  TextField cantidad = new TextField(...);
2  cantidad.addActionListener(new ActionListener<ActionEvent>()
3  {
4      @Override
5      public void actionPerformed(ActionEvent evt)
6      {
7          double cantidad = Double.parseDouble(textCantidad.getText());
8          if( cantidad > 10000 )
9              _descuento.setText("3.00 %");
10         else if( cantidad > 5000 )
11             _descuento.setText("1.50 %");
12         else
13             _descuento.setText("0.00 %");
14     }
15 });

```

Forms and controls

- El cálculo del descuento es **código de negocio**. Debemos mover este cálculo a un método adecuado en las clases de negocio.

Forms and controls

- El cálculo del descuento es **código de negocio**. Debemos mover este cálculo a un método adecuado en las clases de negocio.

```

1  TextField cantidad = new TextField(...);
2  cantidad.addActionListener(new ActionListener<ActionEvent>()
3  {
4      @Override
5      public void actionPerformed(ActionEvent evt)
6      {
7          double cantidad = Double.parseDouble(textCantidad.getText());
8          double descuento = cliente.descuento(cantidad);
9
10         _descuento.setText(Double.toString(descuento));
11     }
12 });
  
```

Forms and controls

- Ventajas:

Forms and controls

- Ventajas:
 1. Se trata de una arquitectura **sencilla** y fácil de implementar.

Forms and controls

- Ventajas:
 1. Se trata de una arquitectura **sencilla** y fácil de implementar.
 2. Toda la lógica de la GUI queda contenida en las clases que implementan cada interfaz.

Forms and controls

- Ventajas:
 1. Se trata de una arquitectura **sencilla** y fácil de implementar.
 2. Toda la lógica de la GUI queda contenida en las clases que implementan cada interfaz.
- Desventajas:

Forms and controls

- Ventajas:
 1. Se trata de una arquitectura **sencilla** y fácil de implementar.
 2. Toda la lógica de la GUI queda contenida en las clases que implementan cada interfaz.
- Desventajas:
 1. El código de interfaz puede quedar demasiado recargado para las interfaces más complicadas.

Forms and controls

- Ventajas:
 1. Se trata de una arquitectura **sencilla** y fácil de implementar.
 2. Toda la lógica de la GUI queda contenida en las clases que implementan cada interfaz.
- Desventajas:
 1. El código de interfaz puede quedar demasiado recargado para las interfaces más complicadas.
 2. La interfaz debe saber en qué momento **cambia el estado** del sistema, para actualizar los controles.

Model-View-Controller

- Separamos la lógica de visualización de la lógica de actualización del sistema.

Model-View-Controller

- Separamos la lógica de visualización de la lógica de actualización del sistema.
 1. **Model:** Las clases de negocio, que deben representar adecuadamente las entidades del dominio del problema en cuestión.

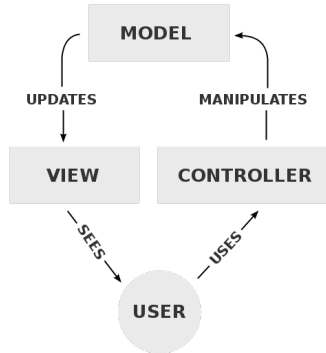
Model-View-Controller

- Separamos la lógica de visualización de la lógica de actualización del sistema.
 1. **Model:** Las clases de negocio, que deben representar adecuadamente las entidades del dominio del problema en cuestión.
 2. **View:** El código encargado de mostrar la información al usuario, y que contiene los controles visuales.

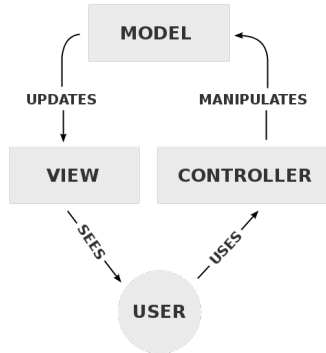
Model-View-Controller

- Separamos la lógica de visualización de la lógica de actualización del sistema.
 1. **Model:** Las clases de negocio, que deben representar adecuadamente las entidades del dominio del problema en cuestión.
 2. **View:** El código encargado de mostrar la información al usuario, y que contiene los controles visuales.
 3. **Controller:** Código encargado de tomar el *input* del usuario y tomar las acciones necesarias sobre ese *input*, llamando al código de negocio.

Model-View-Controller



Model-View-Controller



- Sin embargo, en este diagrama el modelo **actualiza** la vista ... y para eso debe conocerla!

Model-View-Controller

- Solucionamos este problema por medio de un **Observer**.

Model-View-Controller

- Solucionamos este problema por medio de un **Observer**.
- Para cada evento o grupo de eventos importantes, el código de negocio establece un observador:

Model-View-Controller

- Solucionamos este problema por medio de un **Observer**.
- Para cada evento o grupo de eventos importantes, el código de negocio establece un observador:

```

1  public Interface ObservadorPedidos
2  {
3      public void notificar(Pedido pedido);
4  }
```

Model-View-Controller

- Solucionamos este problema por medio de un **Observer**.
- Para cada evento o grupo de eventos importantes, el código de negocio establece un observador:

```

1  public Interface ObservadorPedidos
2  {
3      public void notificar(Pedido pedido);
4  }
```

- Esta declaración puede ser interna a alguna clase de negocio o bien ser una interfaz independiente.

Model-View-Controller

- Solucionamos este problema por medio de un **Observer**.
- Para cada evento o grupo de eventos importantes, el código de negocio establece un observador:

```

1  public Interface ObservadorPedidos
2  {
3      public void notificar(Pedido pedido);
4  }
```

- Esta declaración puede ser interna a alguna clase de negocio o bien ser una interfaz independiente.
- El código de interfaz contiene una o más clases que implementan esta interfaz, y que actualizan la vista en el método notificar().

Model-View-Controller

- Las clases de negocio en las que se producen cambios de estado tienen una lista de observadores:

Model-View-Controller

- Las clases de negocio en las que se producen cambios de estado tienen una lista de observadores:

```

1 public class Pedido
2 {
3     private ArrayList<ObservadorPedidos> observadores;
4     public void registrar(ObservadorPedidos observador)
5     {
6         observadores.add(observador);
7     }
8 }
```

Model-View-Controller

- Cada vez que hay un cambio de estado, el código de negocio **notifica** a los observadores:

Model-View-Controller

- Cada vez que hay un cambio de estado, el código de negocio **notifica** a los observadores:

```

1  public void setCantidad(double cantidad)
2  {
3      _cantidad = cantidad;
4      _descuento = descuento(this);
5      notificarObservadores();
6  }
7
8  private void notificarObservadores()
9  {
10     for(ObservadorPedidos observador: observadores)
11         observador.nodificar(this);
12 }
13 }
```

Model-View-Controller

- El mecanismo de observadores es un caso particular del principio de **inversión de dependencias**.

Model-View-Controller

- El mecanismo de observadores es un caso particular del principio de **inversión de dependencias**.
 1. El modelo no conoce los detalles de la interfaz.

Model-View-Controller

- El mecanismo de observadores es un caso particular del principio de **inversión de dependencias**.
 1. El modelo no conoce los detalles de la interfaz.
 2. Cuando el modelo cambia, la vista es notificada y los controles cambian automáticamente.

Model-View-Controller

- El mecanismo de observadores es un caso particular del principio de **inversión de dependencias**.
 1. El modelo no conoce los detalles de la interfaz.
 2. Cuando el modelo cambia, la vista es notificada y los controles cambian automáticamente.
 3. Se puede tener más de un par *view-controller* sobre el mismo modelo. Todas las vistas se actualizan automáticamente cuando hay cambios, dado que el modelo puede tener más de un observador.

Model-View-Controller

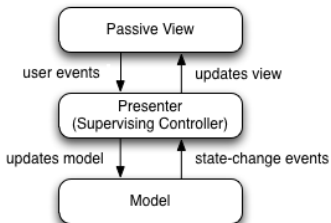
- El mecanismo de observadores es un caso particular del principio de **inversión de dependencias**.
 1. El modelo no conoce los detalles de la interfaz.
 2. Cuando el modelo cambia, la vista es notificada y los controles cambian automáticamente.
 3. Se puede tener más de un par *view-controller* sobre el mismo modelo. Todas las vistas se actualizan automáticamente cuando hay cambios, dado que el modelo puede tener más de un observador.
- A pesar de que “Model-View-Controller” es un término muy utilizado, no es habitual ver implementaciones completas de esta arquitectura.

Model-View-Presenter

- El mecanismo de interacción “circular” de MVC puede ser demasiado estricto.

Model-View-Presenter

- El mecanismo de interacción “circular” de MVC puede ser demasiado estricto.
- MVP es un intento de combinar MVC con *Forms and controls*, dando una **capa de presentación** que combina responsabilidades de vista y controlador.



Model-View-Presenter

- La **vista** es pasiva y solamente contiene controles visuales para mostrar la información. No contiene código para especificar cómo reaccionar ante las acciones del usuario.

Model-View-Presenter

- La **vista** es pasiva y solamente contiene controles visuales para mostrar la información. No contiene código para especificar cómo reaccionar ante las acciones del usuario.
- Cuando se genera un evento sobre un control visual, se pasa el evento a un método adecuado del **presenter**. El presenter decide qué hacer en este caso, y ejecuta los métodos adecuados de la capa de negocio.

Model-View-Presenter

- La **vista** es pasiva y solamente contiene controles visuales para mostrar la información. No contiene código para especificar cómo reaccionar ante las acciones del usuario.
- Cuando se genera un evento sobre un control visual, se pasa el evento a un método adecuado del **presenter**. El presenter decide qué hacer en este caso, y ejecuta los métodos adecuados de la capa de negocio.
- Cuando el **modelo** se modifica, el presenter reacciona y actualiza la vista. El presenter puede ser notificado por medio de observadores (**observer sincronization**), o bien consultando a los métodos adecuados del modelo ante cada cambio relevante (**flow sincronization**).